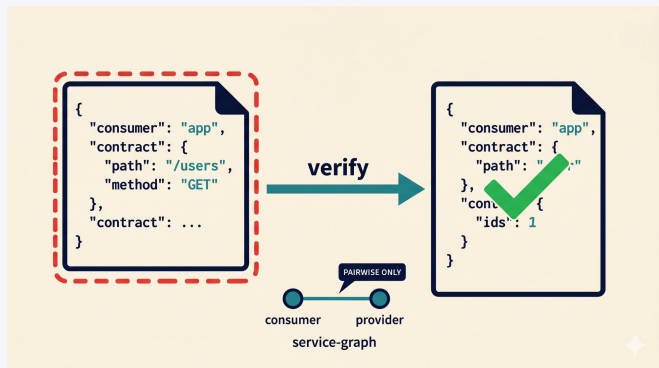


Pact: pairwise contract testing



consumer-expectation.json

```
{
  "consumer": "OrderService",
  "provider": "PaymentService",
  "interactions": [{
    "request": {
      "method": "POST",
      "path": "/charge",
      "body": { "amount": 100,
                "currency": "USD" }
    },
    "response": {
      "status": 200,
      "body": { "txId": "abc-123",
                "status": "approved" }
    }
  }]
}
```

provider-verify-result

```
// Provider v2 response
{
  "status": "approved"
  // txId removed
}

// pact verify output
//
-
// FAIL expected key 'txId'
// in response body
// actual: undefined
// expected: string

// Behavioural regression
// caught for this consumer.
```

CATCHES removed fields · missing required keys · status-code regressions · response shape changes — strictly within the tested examples, for every consumer that maintains a contract.

What Pact misses — and the lattice fix

1

Unknown consumers

A new downstream with no Pact contract has no protection at all.

2

No global composition

Pairwise passes don't aggregate; $A \rightarrow B$ and $B \rightarrow C$ OK $\neq A \rightarrow B \rightarrow C$ OK.

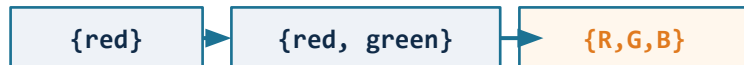
3

Tests cover examples, not types

Provider adds enum value "auditor"; every existing test passes; switch-on-role consumer crashes at runtime.

STRUCTURAL TYPE LATTICE

Enum widening:



Primitive lattice (JSON-lenient):



$T_1 \leq T_2$ iff every value of T_1 is admitted by T_2 .

Caught structurally, regardless of test examples:

request narrowing, response widening, nullable variance, closed objects.

FIX lift type compatibility to a lattice. Enums become subsets; primitives become a partial order. Per-edge subtyping checks every value path, not just the ones the tests happened to hit.

Two relations: $\leq_{\text{req}}$ on the request leg, $\leq_{\text{res}}$ on the response leg

A handful of subtyping rules

ENUM — REQUEST

$\text{values}(\text{caller_sends}) \subseteq \text{values}(\text{provider_accepts})$

Caller may only send enum values the provider admits. Caller-side narrowing OK; provider-side narrowing breaks.

ENUM — RESPONSE (DUAL)

$\text{values}(\text{provider_returns}) \subseteq \text{values}(\text{caller_expects})$

Provider may only return enum values the caller can parse — unless the caller marks x-tolerance.

OBJECT — REQUEST

$\text{required}(\text{provider}) \subseteq \text{sent}(\text{caller}) \cup \text{defaults}(\text{provider})$

Provider can require only fields the caller already sends, or that the provider supplies a default for.

OBJECT — RESPONSE (DUAL)

$\text{required}(\text{caller}) \subseteq \text{returned}(\text{provider})$

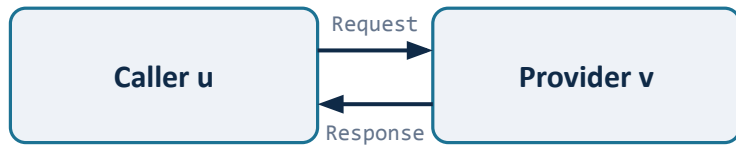
Provider must keep returning every field the caller still requires. Optional $\rightarrow$ required is safe; required $\rightarrow$ optional breaks.

ALSO Nullable variance — $T \leq_{\text{req}} \text{Nullable}(T')$ OK, $\text{Nullable}(T) \leq_{\text{req}} T'$ breaks. Recursive types: coinductive $(\text{Ref}, \text{Ref}) \mapsto \top$. Decidable on finite ASTs.

Separating what the caller sends from what the provider accepts

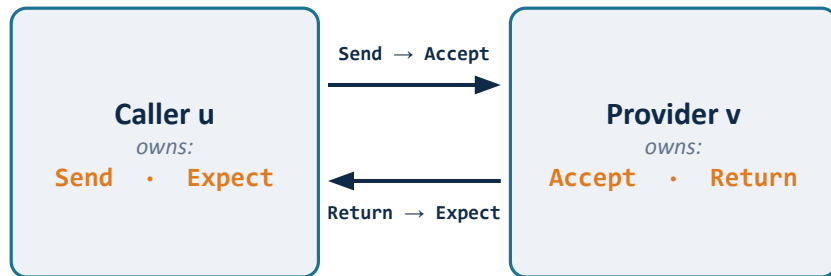
From two schemas to four

TRADITIONAL (2 SCHEMAS)



Assumes **Send** $\equiv$ **Accept**. Caller-side drift is invisible.

FOUR-SCHEMA MODEL



Each schema owned by exactly one service. **Send** $\neq$ **Accept** when caller code drifts.

CODE DRIFT the caller's in-code struct + serializer can diverge from its own OpenAPI / proto spec. Send is derived per tier — Tier 1 source code (highest fidelity), Tier 2 declared spec, Tier 3 default-to-Accept. The 4-schema model makes the gap visible; it doesn't erase it.

Rolling deployment means every cross-version pairing must type-check

Session types: one pairing, then three more

C1 – old caller → new provider

```

u (caller,  $\theta$ )      : ! Send(e,  $\theta$ ) . ? Expect(e,  $\theta$ ) . end
v (provider,  $\theta'$ )   : ? Accept(e,  $\theta'$ ) . ! Return(e,  $\theta'$ ) . end

dual & well-typed channel iff:
    Send(e,  $\theta$ )  $\leq_{\text{req}}$  Accept(e,  $\theta'$ )
  
```

$\leq_{\text{req}}$ subtyping for the request leg (caller's send fits inside provider's accept).

$\leq_{\text{res}}$ subtyping for the response leg (provider's return fits inside caller's expect).

*Borrowed from multiparty session types [Honda et al. POPL'08], applied here as **binary subtyping on each edge in isolation** — no global protocol obligation.*

THE OTHER THREE

C2

new u → old v

Send(e, θ') $\leq_{\text{req}}$ Accept(e, θ)

C3

new v → old u

Return(e, θ') $\leq_{\text{res}}$ Expect(e, θ)

C4

old v → new u

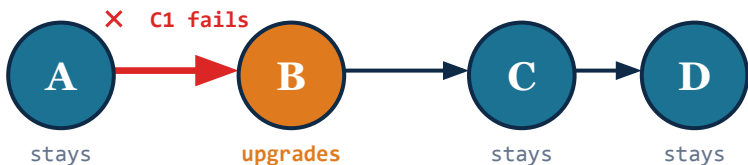
Return(e, θ) $\leq_{\text{res}}$ Expect(e, θ')

WHY ONLY THREE the 4th pairing (θ', θ') — both sides on new code — would generate a non-Horn clause ($\neg x_u \vee \neg x_v$): excluding either of u, v suffices. Putting it inside MSS pushes the problem out of P. Solution: run MSS to a safe subset U' , then call $\text{GUS}(\theta', \theta')$ once as a post-hoc target-state check.

Each upgrade is a boolean; each break is a Horn clause

Per-edge checks become Horn clauses

WORKED EXAMPLE — B UPGRADES



Scenario.

B ships v2; its new Accept narrows an enum on POST /fulfill.

A still ships values from the old enum.

Result: messages from A that B used to accept are now rejected.

TRACING THE CLAUSE

```

Edge A → B, conjunct C1:
  Send(e, θ_A) ≤_req Accept(e, θ_B')

      θ-side (A unchanged)      θ'-side (B upgraded)
                                ↑
                                the moved
schema

  FAILS because new Accept(B) is narrower.

→ emit clause:  ¬ x_B  ∨  x_A
  
```

Reads as: "if B upgrades, A must upgrade too" — so A stops sending values B no longer accepts.

PINNING RULE every EdgeOK conjunct fixes one schema at θ and one at θ' . A failure therefore names exactly one upgrade variable on the θ' side as the cause — yielding a clause with at most one positive literal: Horn.

HORN_MSS: the linear-time safe-subset solver

MAX_SAFE_UPGRADES — pseudocode

```

procedure MAX_SAFE_UPGRADES(clauses, candidates):

    safe ← copy(candidates)      // try to upgrade all
    blocked ← services_in(clauses) - candidates
        U { s : ¬x_s is a unit clause }

    while blocked not empty:
        s ← blocked.pop()
        safe.discard(s)

        // for every clause "¬x_a ∨ x_s",
        // s being blocked means a is too:
        for each clause (¬x_a ∨ x_s) in clauses:
            if a ∈ safe:
                blocked.push(a)

    return safe                    // the maximal safe subset
  
```

HOW IT WORKS

Start every candidate optimistically marked safe.

Block anything outside the candidate set, or forced out by a unit clause.

Propagate if s is blocked, every requirer of s upgrading is blocked too.

Visit each variable is blocked at most once.

Cost $O(|\text{clauses}| + |\text{candidates}|)$. Unique maximal safe set.

OUTPUT the returned set is the unique maximum-cardinality assignment that satisfies every Horn constraint — the largest subset of candidates that ship together safely.

Catching multi-hop integrity violations



CORE IDEA

Source declares **x-provides: K** on the field that originates identity K. Sink declares **x-requires: K** on the field where K must arrive present and non-null.

Intermediates annotate nothing.

Checker discovers the chain by graph traversal + field-name matching across hops. Per-edge type checks can all pass (every hop tolerates optional) while `chain_ok(K)` still fails — exactly the bug class Pact misses.

RESOLVER TIERS

- 1 exact name match
- 2 case-normalised (`correlationId` $\equiv$ `correlation_id`)
- 3 x-alias declaration at a renaming hop

`chain_ok(K,  $\theta'$ )` if sink requires K, every edge in the discovered chain must carry K present and non-null at θ' . Zero annotations on B and C.

Same identity, renamed at one hop — *x-alias* bridges it

Worked example: a chain through a rename



A – order-service.yaml

```

service: order-service
returns:
  correlationId: string

annotations:
  x-provides:
    - field: returns.correlationId
      identity: K_corr
  
```

B – fulfillment.yaml (x-alias)

```

service: fulfillment
calls:
  shipping:
    sends:
      traceId: string # renamed!

annotations:
  x-alias:
    - field: sends.traceId
      previous: correlationId
  
```

D – audit.yaml

```

service: audit
accepts:
  traceId: string

annotations:
  x-requires:
    - field: accepts.traceId
      identity: K_corr
  
```

DISCOVERED CHAIN A.correlationId → B.correlationId → B.traceId (via x-alias) → C.traceId → D.traceId. chain_ok(K_corr) requires present & non-null at every hop.